TASK-6

KRUSKAL ALGORITHM

Given an undirected weighted connected graph, it required to find the Really Special subTree in it. The Really Special SubTree is defined as a subgraph consistin of all the nodes in the graph. Even then if there is collision, Choose any one of them. While doing the above, ensure that no cycle is formed while Picking up edges. Finally, you need to print the overall weight of the Tree so formed using abov rules.

CODING:

```
# include <stdio.h>
# include <stdlib.h>
// Structure to represent an edge
typedef struct{
    int u, v, w;
} Edge;
// Structure for union-find (disjoint set)
typedef struct{
    int parent;
    int rank;
} Node;
// Compare function for sorting edges
int compareEdges(const void*a, const void*b){
    Edge*e1 = (Edge*)a;
    Edge*e2 = (Edge*)b;
if(e1->w!=e2->w){
    return e1->w - e2->w;
}else{
```

```
//If weights are equal, compare by (u+w+v)
        int sum1 = e1->u + e1->w + e1->v;
    int sum2 = e2->u+e2->w+e2->v;
        return sum1- sum2;
        }
    }
// Find operation with path compression
    int find(Node nodes[], int x) {
    if (nodes[x].parent != x){
    nodes[x].parent = find(nodes, nodes[x].parent);
    }
    return nodes[x].parent;
}
  // union operation by rank
void unionNodes(Node nodes[], intx, inty){
      int rootx = find(nodes, x);
      int rootY = find(nodes, y);
if (rootx != rootY){
    if (nodes[rootX].rank < nodes[rootY].rank){
        nodes[rootx].parent = rootY;
} else if (nodes[rootx].rank > nodes[rootY].rank){
      nodes[rootY].parent = rootx;
    } else {
        nodes[rootY].parent = rootx;
      nodes[rootx].rank++;
      }
    }
  }
   int kruskalMST(Edge edges[], int N, int M){
  //Sort edges according to custom rules
```

```
q sort (edges, M, Sizeof(Edge), compareEdges);
// intialize union find structure
Node* nodes = (Node*) malloc((N+1)* sizeof(Node));
  for (int i=1; i<=N; i++){
      nodes[i].parent = i;
      nodes[i]. rank = 0;
  }
      int totalWeight =0;
      int edgesAdded = 0;
// Process edges in sorted order
for (int i=0; i< M&& edgesAdded < N-1; i++){
    Edge e = edges[i];
// Ensure u <= v for consistency in tie breaking
      if (e.u > e.v) {
          int temp = e.u;
            e.u = e.v;
            e.v = temp;
        }
      int root U = find(nodes, e.u);
      int root v = find(nodes, e.v);
// If adding this edge doesn't create a cycle
        if (root U != root v){
        totalWeight += e.w;
        edges Added++;
        unionNodes(nodes, root U, root V);
        }
      }
        free(nodes);
        return totalWeight;
      }
```

```
int main(){
  int N, M;
Scanf("%d %d", &N, &M);
Edge *edges = (Edge*) malloc(M* size of(Edge));
    for (int i=0; i<M; i++){
Scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].w);
  //Store with smaller node first for consistency
     if(edges[i].u > edges[i].v){
          int temp = edges[i].u;
        edges[i].u = edges[i].v;
        edges[i].v = temp;
         }
       }
         int result = kruskalMST(edges, N, M);
Printf("%d\n", result);
    free(edges);
       return 0;
    }
```

VEL TECH - CSE	
EX NO.	8
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	5
RECORD (5)	5
TOTAL (20)	20
SIGN WITH DATE	14/12/24

RESULT: For the given undirected weighted connected graph the minimum spanning tree is identified using kruskal algorithm was executed and verified sucessfully.